UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

Materia

SOA

Manual Técnico

Librería Cartagena — tienda en línea

Arquitectura, despliegue y mantenimiento para el equipo de desarrollo.

Integrantes

Nombre	Número de cuenta
Sarahi Yamileth García Bojórquez	23020009
Aranzazú Acosta Medina	23021126
Alan Zayas Maldonado	23020112
Cesar Omar Karass Cota	23020143

Fecha de creación: 03 de noviembre de 2026

Documento académico — Materia SOA · Librería Cartagena

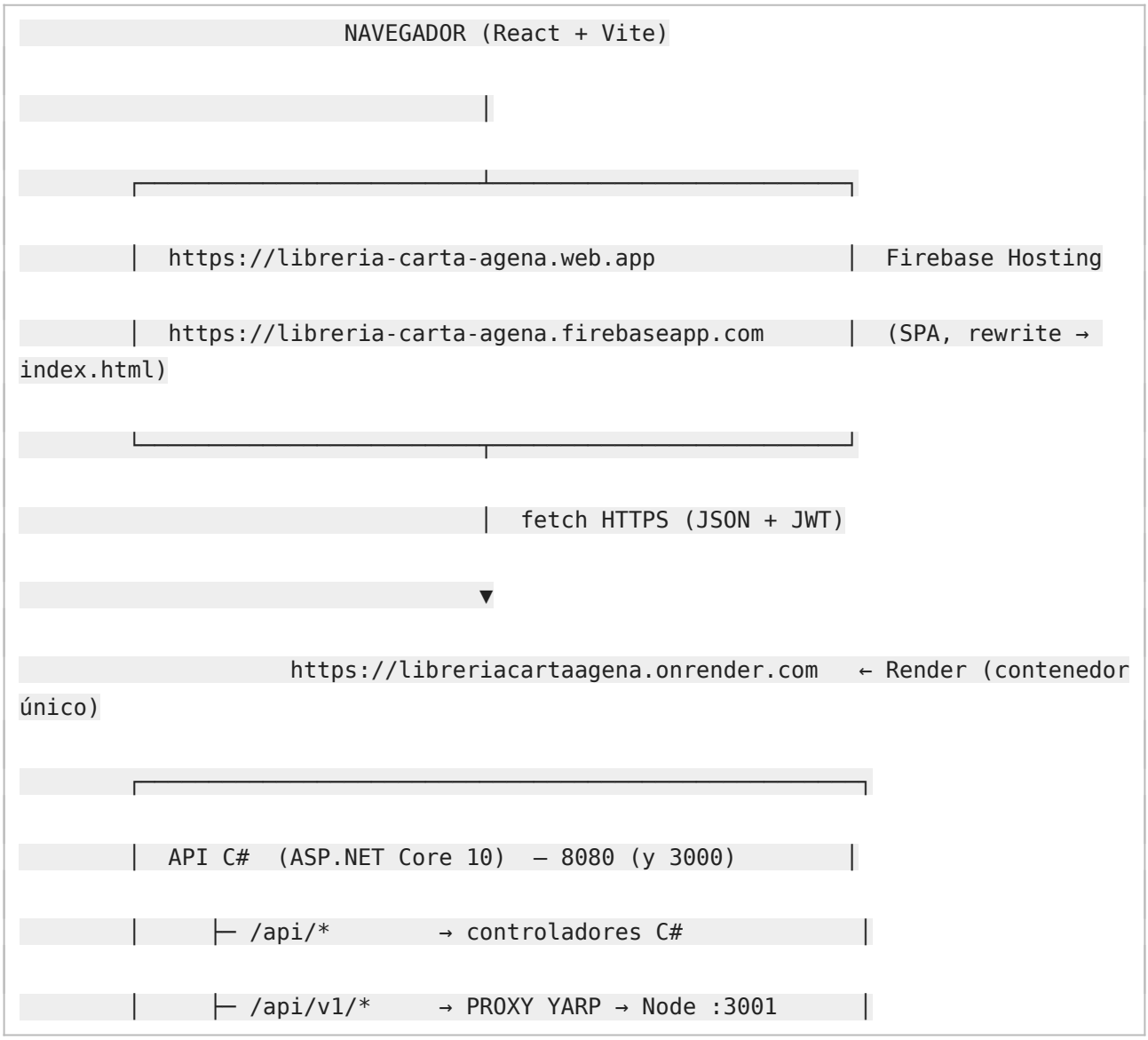
Manual Técnico — Librería Cartagena

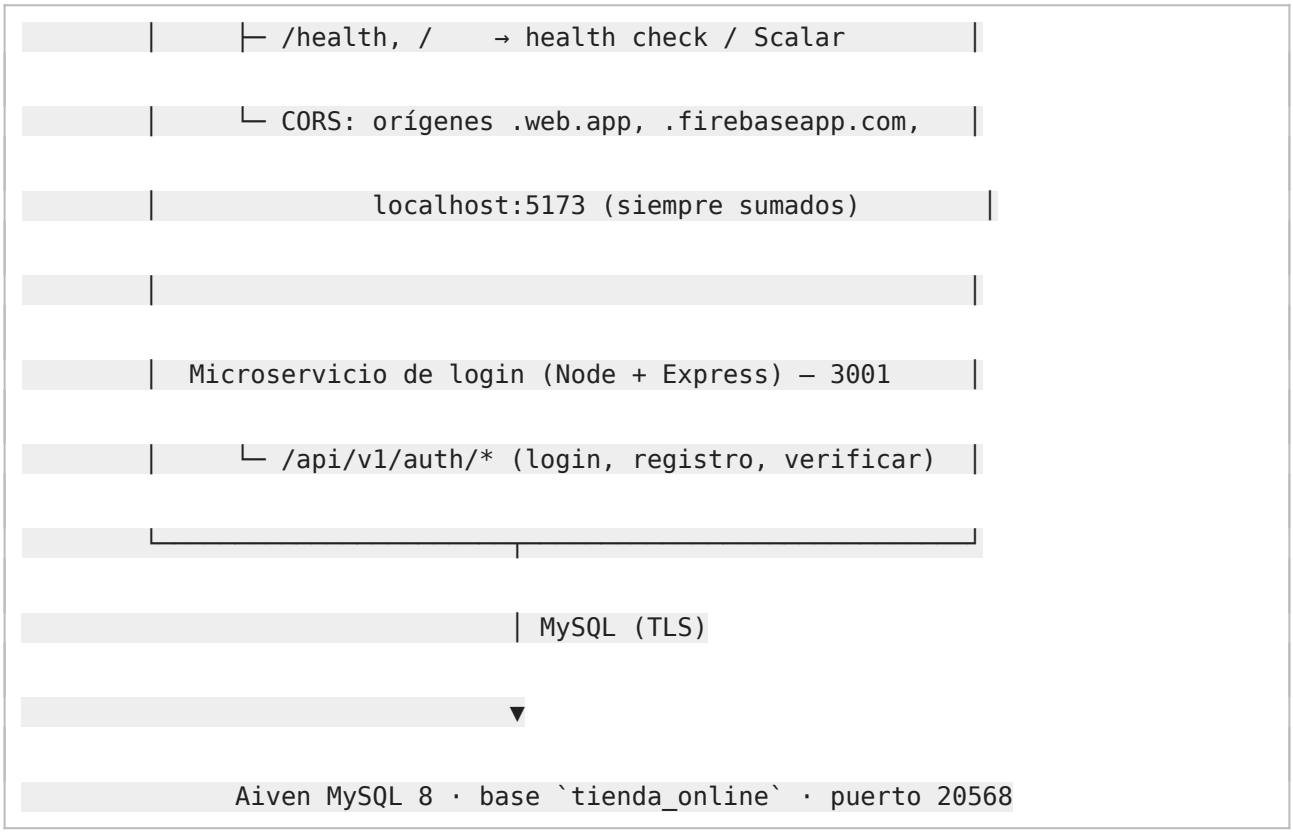
Documento para el equipo de desarrollo y mantenimiento. Describe la arquitectura, los repositorios, las variables de entorno, los despliegues y la solución de problemas del sistema **Cartagena Bookstore**.

> Compañero de este documento: `docs/Manual-de-Usuario.md` (uso final).

---

1. Arquitectura





1.1 Topología

Entorno	Frontend	API C#	Login	Base de datos
Producción	Firebase Hosting	Render (`libreriacartaagena.onrender.com`)	**Dentro del mismo contenedor** (puerto interno 3001, publicado por el proxy YARP)	Aiven MySQL (`mysql-3da06998-…:20568`)
Local (nativo)	Vite `:5173`	`:5000`	`:3000`	`mysql-cartagena` en `127.0.0.1:3307`
Local (Docker)	Vite `:5173`	`:8083` (gateway `:8081`)	`:3000`	contenedor `mysql` en la red `libreria-network`

Regla de oro: el frontend nunca habla con MySQL; solo con HTTP (`/api/*` para la API C# y `/api/v1/*` para el login).

---

2. Repositorios

Repositorio	Rama	Uso
`https://github.com/CesarKarass/LibreriaCartagena`	`Frontend`	Repositorio principal del equipo (colaboradores).
`https://github.com/Yami678/LibreriaCartaAgena`	`main`	**Desde aquí despliega Render** (copia sincronizada del backend).

El trabajo se hace en el repositorio principal y se sincroniza al de Render:

```
TOKEN=<con permisos de escritura en ambos>

# principal

git push
"https://x-access-token:${TOKEN}@github.com/CesarKarass/LibreriaCartagena.git"
Frontend:Frontend



# repositorio de Render (solo Backend/Cartagena)

rsync -a --exclude bin/ --exclude obj/ --exclude node_modules/ \

      --exclude backups/ --exclude .env \

      Backend/Cartagena/ /tmp/opencode/yami678/Backend/Cartagena/

cd /tmp/opencode/yami678 && git add -A && git commit -m "…" && \

  git push
"https://x-access-token:${TOKEN}@github.com/Yami678/LibreriaCartaAgena.git"
main:main
```

> **Nunca** sincronizar `.env`: está en `.gitignore` y contiene credenciales.

3. Componentes y puertos

Componente	Carpeta	Puerto	Notas
Frontend React + Vite	`Frontend/`	`5173` (dev)	Build estático a `dist/`.
API C# (ASP.NET Core 10)	`Backend/Cartagena/backend/Cartagena.Api/`	`5000` local · `8080` y `3000` en el contenedor	Dapper + MySqlConnector.
Microservicio de login	`Backend/Cartagena/servicios/microservicio-login/` (copia embebida en `Cartagena.Api/login/`)	`3000` local · `3001` en el contenedor	Express + JWT + bcrypt.
MySQL local	contenedor `mysql-cartagena`	`3307`	Base `tienda_online`, usuario `usuario_app`.
MySQL producción	Aiven	`20568`	Usuario `avnadmin`, TLS.
Flask de proveedores (legado)	`Backend/Cartagena/servicios/proveedores/`	`5000/5001`	**Ya no se usa**: su CRUD fue reimplementado en `ProveedoresController.cs`.

3.1 El contenedor de producción (único servicio)

`Dockerfile` de `Cartagena.Api`:

1. Etapa de build: .NET SDK 10 → `dotnet publish`.
2. Etapa final: aspnet 10 **+ Node.js 20** (descargado de nodejs.org).
3. Copia el microservicio a `/app/login` y ejecuta `npm ci --omit=dev`.
4. `ENTRYPOINT /entrypoint.sh`: - arranca el login Node en `PORT=3001` (proceso secundario); - `exec dotnet Cartagena.Api.dll` (proceso principal, recibe SIGTERM de Render).
5. `ENV ASPNETCORE_URLS=http://+:8080;http://+:3000` y **solo** `EXPOSE 8080`.

> ⚠️ **No exponer el 3000 como puerto del login.** Render enruta el tráfico al puerto que anuncia la imagen: si se publica el puerto del Node, *todo* el tráfico llega al Express y la API C# deja de responder (se perdían `/health`, `/api/*` y el proxy). El 3000 del contenedor es de la **API C#**, a modo de respaldo.

El proxy inverso está declarado en `Program.cs` con **YARP**:

```
Route: /api/v1/{**catch-all}  →  destino: LOGIN_PROXY_TARGET
(http://localhost:3001)
```

`login/LEEME.md` explica cómo volver a copiar el microservicio original si cambia.

---

4. Variables de entorno

4.1 API C# / contenedor (Render)

Variable	Ejemplo / valor en producción	Descripción
`DB_HOST`	`mysql-3da06998-…i.aivencloud.com`	Host MySQL.
`DB_PORT`	`20568`	Puerto MySQL.
`DB_NAME`	`tienda_online`	Base de datos.
`DB_USER`	`avnadmin`	Usuario MySQL.
`DB_PASSWORD`	*(secreto en Render)*	Contraseña MySQL.
`JWT_SECRET`	*(secreto, ≥ 32 bytes)*	Firma HS256. **Debe ser idéntico al del login.**
`JWT_ISSUER`	`microservicio-login`	Emisor del token.
`JWT_EXPIRES_IN`	`15m`	Vigencia del token (afecta la duración de sesión).
`CORS_ORIGIN`	`https://libreria-carta-agena.web.app,…`	Orígenes extra. Los dominios `.web.app` y `.firebaseapp.com` **se suman siempre** en código.

`LOGIN_PROXY_TARGET`	`http://localhost:3001`	Destino del proxy YARP (opcional).
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASSWORD` / `SMTP_FROM`	*(vacíos hoy)*	Envío de correos. Ver §8.
`RECAPTCHA_SECRET`	*(vacío)*	Verificación server-side del reCAPTCHA.

4.2 Frontend (`Frontend/.env.production`)

Variable	Valor en producción
`VITE_API_URL`	`https://libreriacartaagena.onrender.com/api`
`VITE_AUTH_API_URL`	`https://libreriacartaagena.onrender.com/api/v1`
`VITE_RECAPTCHA_SITE_KEY`	site key de **prueba** de Google (ver §7)
`VITE_USE_MOCKS`	`false`

`.env.development` usa rutas relativas (`/api`, `/api/v1`) y el proxy de Vite.

---

5. API

Documentación interactiva: **`/scalar/v1`** (Scalar/OpenAPI) en el servidor.

5.1 Login (microservicio Node, vía proxy)

Método	Ruta	Descripción
POST	`/api/v1/auth/login`	Body `{"nombreUsuario","contrasena"}` → token JWT.
POST	`/api/v1/auth/registro`	Alta de cuenta.
GET	`/api/v1/auth/usuario-disponible?nombreUsuario=`	Consulta de duplicados (registro).

GET	`/api/v1/auth/verificar`	Valida el token (`Authorization: Bearer`).
GET	`/api/v1/auth/admin/panel`	Confirma el rol ADMIN (panel).
GET	`/api/v1/estado`	Estado del microservicio.

5.2 API C# (selección)

Método	Ruta	Rol
GET	`/api/libros`, `/api/libros/{id}`	público
GET/POST/ PUT/DELETE	`/api/proveedores`, `/api/proveedores/{id}`	ADMIN
GET/POST	`/api/compras-proveedor`, `…/{id}/recibir`	ADMIN
GET/POST	`/api/admin/inventario`, `/api/admin/usuarios`, `/api/admin/roles`	ADMIN
GET	`/api/admin/pedidos`, `/api/admin/resumen`, `/api/admin/logistica/envios`	ADMIN
POST	`/api/checkout`	autenticado
GET	`/api/mis-compras`, `/api/ventas/{id}/ticket.pdf`	autenticado
POST	`/api/cuenta/recuperacion/solicitar`, `…/restablecer`	público
GET	`/health`	público (health check de Render)

Convención de respuestas

- Éxito: `{ "success": true, "message"?: string, "data": … }` (snake_case).
- Error: `{ "error": "mensaje" }` con el código HTTP correspondiente (400 validación · 401 sin token · 403 rol insuficiente · 404 inexistente · 409 conflicto · 503 servicio caído o no configurado). El manejador global es `Infrastructure/ApiExceptionHandler.cs`.

5.3 Autorización

Políticas definidas en `Program.cs`:

Política	Roles aceptados
`Admin`	`ADMIN`
`Staff`	`ADMIN`, `REPARTIDOR`
`Comprador`	`CLIENTE`, `ADMIN`

El claim se lee de `rol` en el JWT emitido por el microservicio de login.

5.4 Módulo de proveedores (contrato)

`Controllers/ProveedoresController.cs` replica el contrato del antiguo Flask:

Método	Ruta	Éxito	Errores
GET	`/api/proveedores`	200 `{success,data}`	401/403
GET	`/api/proveedores/{id}`	200	404
POST	`/api/proveedores`	201 `{success,message,data}`	400 (faltan llaves/vacío), 409 (RFC duplicado)
PUT	`/api/proveedores/{id}`	200	400 (sin campos), 404, 409 (RFC de otro)
DELETE	`/api/proveedores/{id}`	200	404, 409 (tiene compras)

Reglas de validación idénticas al Flask original: en el **alta** se exigen las siete llaves (aunque el valor sea `null`), en la **edición** basta con un campo; `nombre` es obligatorio y no puede ir vacío; se respetan los largos de columna (150/150/150/13/255/150/50) para no provocar errores 1406.

---

6. Base de datos

Entorno	Conexión
Local	`127.0.0.1:3307`, base `tienda_online`, usuario `usuario_app`, contraseña en `Backend/Cartagena/.env`.

Producción	Aiven MySQL, `…i.aivencloud.com:20568`, usuario `avnadmin`, base `tienda_online`.

Tablas principales: `rol`, `usuario`, `token_cuenta`, `libro`, `carrito`, `item_carrito`, `venta`, `detalle_venta`, `domicilio`, `envio`, `paquete`, `pedido_compra`, `detalle_pedido_compra`, `proveedor`, `devolucion_proveedor`, `movimiento_inventario`, `comentario`, `solicitud_garantia`, `tipo_pago`.

Sincronizar datos a Aiven (respaldo `tienda_online.sql`):

```
H=mysql-3da06998-….i.aivencloud.com

mysql -h $H -P 20568 -u avnadmin -p"$PW" --ssl-mode=REQUIRED \

  -e "DROP DATABASE IF EXISTS tienda_online; CREATE DATABASE tienda_online \

      CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;"

mysql -h $H -P 20568 -u avnadmin -p"$PW" --ssl-mode=REQUIRED \

  --init-command="SET FOREIGN_KEY_CHECKS=0" --default-character-set=utf8mb4 \

  tienda_online < tienda_online.sql
```

El `--init-command="SET FOREIGN_KEY_CHECKS=0"` es **obligatorio**: el dump ordena las tablas de forma que una clave foránea apunta a una columna aún no recreada (error `3780`).

---

7. reCAPTCHA

- Widget **reCAPTCHA v2 (checkbox)** con `hl=es`, montado en `Frontend/src/components/auth/Recaptcha.tsx`.
- La site key actual es la de **prueba** de Google (`6LeIxAcTAAAAAJcZVRqyHh71UMIEGNQ_MXjiZKhI`): el widget muestra una franja roja de aviso y la clase `.auth-captcha__marco--prueba` recorta el marco (height 55px / translateY −20px) para que no rompa el diseño.
- **Al poner la site key real** de un dominio verificado, esa clase deja de aplicarse automáticamente (`esClaveDePrueba` es falsa) y el widget se dibuja con su tamaño estándar. No hay que tocar el CSS.
- La secreta (`RECAPTCHA_SECRET`) aún no está configurada en el servidor; sin ella el backend no valida el token, solo el frontend exige que exista.

8. Correo (SMTP) — pendiente de configurar

`Services/MailService.cs` exige **las cuatro variables** `SMTP_HOST`, `SMTP_USER`, `SMTP_PASSWORD` y `SMTP_FROM`. Si faltan, la recuperación de contraseña responde de forma honesta:

```
HTTP 503

{"codigo":"SMTP_NO_CONFIGURADO_O_ERROR",

 "mensaje":"El servicio de correo no está configurado, así que no pudimos enviar el token. Completa SMTP_HOST, SMTP_USER, SMTP_PASSWORD y SMTP_FROM en Backend/Cartagena/.env y reinicia el backend."}
```

El token **sí se genera y se registra** en la tabla `token_cuenta` (20 minutos), y el flujo completo (generar → usar una sola vez → cambiar contraseña → entrar) está verificado en `PRUEBAS-ESTRUCTURALES.md` (pruebas 1.7a–1.7e).

Para activar el envío con Gmail (`libreriacartaagena302987@gmail.com`):

1. Cuenta de Google → **Seguridad** → **Verificación en dos pasos** → **activada**.
2. Seguridad → **Contraseñas de aplicación** → generar una para «Otra».
3. En Render (Environment del servicio) agregar:

Variable	Valor
`SMTP_HOST`	`smtp.gmail.com`
`SMTP_PORT`	`587`
`SMTP_USER`	`libreriacartaagena302987@gmail.com`
`SMTP_PASSWORD`	*contraseña de aplicación*
`SMTP_FROM`	`libreriacartaagena302987@gmail.com`

4. Guardar → Render redespliega. Verificar con:

```
bash curl -X POST https://libreriacartaagena.onrender.com/api/cuenta/recuperacion/solicitar \ -H "Content-Type: application/json" -d '{"Correo":"tu-correo@gmail.com"}'
```

Respuesta esperada: **200** con *«Si el correo existe, enviaremos instrucciones»*.

> Mismo `.env` local para probar en desarrollo (`Backend/Cartagena/.env`).

---

9. Despliegues

9.1 Frontend → Firebase Hosting

```
cd Frontend

npm run build

firebase deploy --only hosting --project libreria-carta-agena
```

`firebase.json`:

- **Rewrite** `**` → `/index.html` (SPA).
- **Cabeceras** (la última regla que coincide gana): | Origen | Cache-Control | |---|---| | `**` | `no-cache` (los documentos HTML nunca quedan obsoletos) | | `**/*.@(js\|css)` | `public, max-age=3600` | | `/assets/**` | `public, max-age=31536000, immutable` |

> ⚠️ Antes solo `/index.html` tenía `no-cache`; Firebase **no** aplica esa regla a las rutas SPA, así que el navegador guardaba el HTML hasta 1 h y, tras cada despliegue, pedía un bundle ya borrado → **página en blanco** con el error *"Failed to load module script … MIME type text/html"*. Con `** → no-cache` el navegador siempre revalida (ETag 304) y el problema desaparece. Si vuelve a verse en blanco: **Ctrl + F5**.

9.2 Backend → Render

1. Sincronizar `Backend/Cartagena` al repo `Yami678/LibreriaCartaAgena` (§2).
2. Push a `main` → Render construye la imagen con el `Dockerfile` y la publica.
3. Health check: `GET /health` → `{"status":"ok"}`.

Verificación posterior (debe responder todo):

```
B=https://libreriacartaagena.onrender.com
```

```
curl -s $B/health

curl -s "$B/api/libros?pagina=1" | head -c 200

curl -s -o /dev/null -w "%{http_code}\n" -X POST $B/api/v1/auth/login \

  -H "Content-Type: application/json" -d
'{"nombreUsuario":"admin","contrasena":"ClaveDemo123"}'

curl -s -D - -o /dev/null "$B/api/libros" -H "Origin: https://libreria-carta-
agena.firebaseapp.com" \

  | grep -i access-control-allow-origin
```

9.3 Credenciales demo

Usuario	Contraseña	Rol
`admin`	`ClaveDemo123`	ADMIN
`cliente`	`ClaveDemo123`	CLIENTE
`vendedor`	`ClaveDemo123`	VENDEDOR (sin panel)
`sgarcia1859`	`NuevaClave123!`	ADMIN (cuenta del equipo)

---

10. Desarrollo local

```
./start-dev.sh      # MySQL + login + API C# + Vite (nativo)

./start-docker.sh   # stack completo con Docker

./stop-dev.sh       # detener todo
```

Servicio	URL
Frontend	http://localhost:5173

Login	http://localhost:3000/api/v1
API C#	http://localhost:5000/api
Scalar	http://localhost:5000/scalar/v1
MySQL	127.0.0.1:3307 / `tienda_online`

```
# Compilar solo la API

dotnet build Backend/Cartagena/backend/Cartagena.Api/Cartagena.Api.csproj



# Probar el contenedor completo contra Aiven (puerto de prueba)

cd Backend/Cartagena/backend/Cartagena.Api && docker build -t cartagena-api-login:local .

docker run --rm -p 18080:8080 \

  -e DB_HOST=… -e DB_PORT=20568 -e DB_NAME=tienda_online -e DB_USER=… \

  -e DB_PASSWORD=… -e JWT_SECRET=… -e JWT_ISSUER=microservicio-login \

  -e CORS_ORIGIN=https://libreria-carta-agena.web.app cartagena-api-login:local
```

---

11. Pruebas

`PRUEBAS-ESTRUCTURALES.md` (raíz del repo): **45/45 PASS**, cubriendo autenticación y seguridad, flujo de compra, inventario, logística, panel administrativo y UI (más Lighthouse: accesibilidad 96 %, buenas prácticas 100 %).

Pruebas unitarias del login: `npm test` dentro de `servicios/microservicio-login` (Node test runner + supertest).

---

12. Solución de problemas

Síntoma	Causa	Solución
Página en blanco + *"Failed to load module script"*	HTML cacheado apuntando a un bundle borrado	Ctrl + F5. Causa raíz corregida en `firebase.json` (`** → no-cache`).
`/api/v1/*` responde 404 con errores de Express	El Node está capturando todo el tráfico (puerto mal expuesto)	Solo `EXPOSE 8080`; la API C# debe ser la entrada.
`/api/proveedores` → 404	Despliegue anterior sin `ProveedoresController`	Redesplegar (§9.2).
403 en proveedores	Token de un rol distinto a `ADMIN`	Usar cuenta ADMIN (§9.3).
401 en todo	`JWT_SECRET` distinto entre login y API	Ambos leen el mismo `.env`; verificar en Render.
`RUTA_NO_ENCONTRADA` en `/health`	Estás hablando al Express, no al C#	Revisar puertos/`EXPOSE` del `Dockerfile`.
503 `SMTP_NO_CONFIGURADO_O_ERROR`	Sin variables SMTP	§8.
Aiven: error `3780` al importar	FKs activas durante el dump	`--init-command="SET FOREIGN_KEY_CHECKS=0"`.
CORS bloqueado desde `.firebaseapp.com`	Origen no listado	`Program.cs` suma siempre `.web.app` y `.firebaseapp.com`; revisar `CORS_ORIGIN`.
Login local: 503	MySQL (`3307`) apagado	`docker start mysql-cartagena`.
Puerto ocupado (3000/5000/5173)	Proceso anterior colgado	`./stop-dev.sh`.
